

Hướng dẫn chi tiết: Quy trình Data Task trên OULAD

Hiểu dữ liệu lớn **10,6 triệu dòng** bằng **ClickHouse** — rồi làm sạch, biến đổi, chia tập và phân tích

DSP391m · Nhóm 1 · FPT University

Đề tài: Học máy giải thích được, nhận biết theo thời gian, phát hiện sớm sinh viên nguy cơ (OULAD)

Báo cáo 2 — Data Tasks

Vì sao bước này? Tài liệu này để **đọc và hiểu toàn bộ tiến trình** của phần Data Task — đặc biệt là *cách thật sự hiểu một bảng 10,6 triệu dòng* mà không thể mở bằng Excel hay nạp hết vào pandas. Mỗi bước đều có **lệnh chạy được** và **con số thật**; bản chạy tay từng bước nằm ở `notebooks/00_data_understanding.ipynb`. Mọi con số trong tài liệu khớp với `reports/data_understanding/verified_numbers.json` và tái lập được.

Contents

1	Bức tranh tổng thể: data task gồm những gì?	2
2	Vì sao phải dùng một “engine” cho dữ liệu lớn	2
2.1	Vấn đề: không thể “nạp hết vào RAM” mỗi lần hỏi	2
2.2	Giải pháp: để dữ liệu nằm yên, đẩy câu hỏi (SQL) xuống engine	2
2.3	Cài đặt và cầu nối Python (một lần)	3
3	Hiểu dữ liệu — 11 bước thực hành bằng ClickHouse	3
3.1	Bước 1 — Tiếp xúc đầu tiên & cái bẫy kiểu dữ liệu	3
3.2	Bước 2 — Khai báo kiểu tường minh → con số đúng	3
3.3	Bước 3 — Tăng tốc: CSV → Parquet có kiểu (một lần)	3
3.4	Bước 4 — Hồ sơ <code>studentVle</code>	4
3.5	Bước 5 — Phân phối click & độ lệch phải → vì sao <code>log1p</code>	4
3.6	Bước 6 — Nhân & cân bằng lớp	4
3.7	Bước 7 — Dữ liệu thiếu & cơ chế thiếu	4
3.8	Bước 8 — Các bảng nhỏ còn lại (một câu mỗi bảng)	5
3.9	Bước 9 — Ghép về hạt master ngay trong SQL	5
3.10	Bước 10 — Cắt theo thời gian: trái tim của chống rò rỉ	5
3.11	Bước 11 — Tín hiệu: yếu tố nào phân biệt nguy cơ, và <i>sớm tới đâu?</i>	5
4	Từ dữ liệu đã hiểu → hoàn thành các task còn lại	6
4.1	Làm sạch (Task 3)	6
4.2	Biến đổi & chuẩn hoá (Tasks 16–20)	6
4.3	Chia tập & chống rò rỉ (Tasks 7, 8, 15, 22)	6
4.4	Phân tích khám phá — EDA (Tasks 27–38)	7
5	Tái lập toàn bộ (chạy lại từ đầu)	7
	Phụ lục — Bảng tổng hợp con số	8

1 Bức tranh tổng thể: data task gồm những gì?

Phần Data Task của đề án không phải một việc, mà là một **chuỗi sáu việc** nối tiếp, biến bảy file CSV thô của OULAD thành dữ liệu sẵn sàng cho mô hình:

1. **Dữ liệu cần thiết** — xác định cần bảng nào, cột nào, vì sao.
2. **Thu thập** — vì sao chọn dữ liệu công khai OULAD (tái lập, so sánh được).
3. **Làm sạch** — trùng lặp, giá trị thiếu, ngoại lai.
4. **Biến đổi & chuẩn hoá** — mã hoá, scale, đưa về ma trận số.
5. **Chia tập & chống rò rỉ** — train/test theo nhóm, phân tầng, theo thời gian.
6. **Phân tích khám phá (EDA)** — dữ liệu *nói* gì, dẫn đường cho mô hình.

Đường đi của dữ liệu (pipeline):

7 CSV thô → [gộp + làm sạch] → master_raw (32.593×33) → [cắt theo thời gian] → 6 checkpoint → [tiền xử lý + chia tập] → ma trận 49 cột

Hai khái niệm xuyên suốt cần nắm trước:

- **Hạt dữ liệu (grain):** mỗi dòng cuối cùng = **một sinh viên trong một lượt mở môn** (khóa (code_module, code_presentation, id_student)). Cho ra **32.593** dòng, **28.785** sinh viên, **22** lượt mở môn.
- **Nhãn (label):** at_risk = 1 nếu final_result ∈ {Fail, Withdrawn}, ngược lại 0. Cố định ở mọi mốc thời gian.

⇒ *Hiểu hạt dữ liệu và nhãn trước, rồi mọi bước sau mới có nghĩa.*

2 Vì sao phải dùng một “engine” cho dữ liệu lớn

Bảng nặng nhất, studentVle (clickstream — log mỗi lần sinh viên bấm vào một tài nguyên), có **10.655.280 dòng** ≈ **433 MB**. Đây là chỗ trực giác hay sai.

2.1 Vấn đề: không thể “ nạp hết vào RAM” mỗi lần hỏi

Mở bằng Excel: bất khả thi (Excel trần 1 triệu dòng). Còn pandas.read_csv cả file thì mỗi câu hỏi nhỏ cũng phải đọc lại 433 MB và phình ra nhiều lần thế trong RAM. Đọc thử 500.000 dòng đầu rồi ngoại suy (ô đầu tiên trong notebook) cho thấy: nạp toàn bộ tốn hàng chục giây và **nhiều GB RAM cho mỗi lần thử**. Khi khám phá dữ liệu ta hỏi hàng chục câu — cách này sụp đổ.

2.2 Giải pháp: để dữ liệu nằm yên, đẩy câu hỏi (SQL) xuống engine

Công cụ phân tích dữ liệu lớn được sinh ra cho đúng việc này. Ý tưởng chung: **dữ liệu ở trên đĩa theo định dạng cột, ta gửi một câu SQL xuống, engine quét có chọn lọc và chỉ trả về kết quả đã tóm tắt** (vài dòng). Python không bao giờ phải giữ 10 triệu dòng.

Cách làm	Đặc điểm	Phù hợp?
pandas (nạp cả file)	Đơn giản nhưng tốn RAM, đọc lại mỗi lần	Không, với 10,6M dòng
ClickHouse-local	1 file thực thi, không cần server, SQL thẳng trên CSV/-Parquet, cực nhanh	Có — ta chọn cái này
Apache Spark	Mạnh cho phân tán nhiều máy; nặng, khởi động chậm	Quá mức cho 1 laptop

Nhóm chọn **ClickHouse-local**: nó là một engine cột (OLAP) chạy SQL chuẩn, không cần dựng server, đọc trực tiếp CSV/Parquet, và xử lý 10,6 triệu dòng trong *dưới một giây* sau khi đã chuyển sang Parquet. SQL cũng dễ trình bày trên slide.

2.3 Cài đặt và cầu nối Python (một lần)

ClickHouse chạy trên Linux; trên Windows ta dùng **WSL** (Ubuntu). Cài bằng một dòng:

```
# In WSL Ubuntu -- download a single self-contained binary into home:
curl https://clickhouse.com/ | sh # creates ~/clickhouse
~/clickhouse local --query "SELECT version()" # check: 26.6.x
```

Từ notebook (kernel dsp trên Windows) ta gọi sang WSL bằng một hàm nhỏ: ghi câu SQL ra file, chạy clickhouse local, lấy kết quả CSV nạp vào pandas để hiển thị. Điểm cốt lõi: **Python chỉ gửi câu hỏi và nhận về vài dòng kết quả** (xem ô “Thiết lập” trong notebook).

Vì sao bước này? Đây chính là câu trả lời cho thắc mắc ban đầu: “data 10 triệu dòng thì phải làm sao?”. Câu trả lời là **query**, không phải mở file. Toàn bộ phần sau là minh hoạ cách query để thực sự hiểu dữ liệu.

3 Hiểu dữ liệu — 11 bước thực hành bằng ClickHouse

Mỗi bước dưới đây: *mục đích* → *lệnh* → *kết quả thật* → *ý nghĩa*. Đây là phần “show cách làm” để ai cũng tự kiểm chứng được.

3.1 Bước 1 — Tiếp xúc đầu tiên & cái bẫy kiểu dữ liệu

Việc đầu tiên với một file lạ: hỏi “các cột kiểu gì?”. Vì CSV để giá trị trong ngoặc kép, ClickHouse đoán mọi cột là chuỗi (String).

```
DESCRIBE file('studentVle.csv', 'CSVWithNames');
-- => every column is inferred as Nullable(String)!
SELECT min(date) AS min_str, max(date) AS max_str
FROM file('studentVle.csv', 'CSVWithNames');
-- => max = "99" (WRONG: lexicographic compare, "99" > "269" since '9' > '2')
```

⇒ Bài học #1: với dữ liệu lớn, đừng tin suy đoán kiểu mặc định. `max(date)` ra “99” là vô lý — phải khai báo kiểu.

3.2 Bước 2 — Khai báo kiểu tường minh → con số đúng

Khai báo schema ngay trong hàm `file()`:

```
SELECT count() AS rows, uniqExact(id_student) AS students,
       min(date) AS min_day, max(date) AS max_day, round(avg(sum_click),3) AS
       avg_click
FROM file('studentVle.csv', 'CSVWithNames',
         'code_module String, code_presentation String, id_student Int32,
         id_site Int32, date Int16, sum_click Int32');
```

Kết quả thật: **10.655.280** dòng, **26.074** sinh viên, date chạy từ **-25** (trước khai giảng) tới **269** (độ dài môn), trung bình **3,72** click/sự kiện.

3.3 Bước 3 — Tăng tốc: CSV → Parquet có kiểu (một lần)

CSV phải phân tích lại 433 MB văn bản mỗi lần. **Parquet** là định dạng cột, có kiểu, nén sẵn. Chuyển một lần:

```
INSERT INTO FUNCTION file('studentVle.parquet', 'Parquet')
SELECT * FROM file('studentVle.csv', 'CSVWithNames', '...schema...')
SETTINGS engine_file_truncate_on_insert=1;
```

Hiệu quả đo được:

	Kích thước	Thời gian đếm count()
CSV	433 MB	≈ 14,6 s
Parquet	22,9 MB	≈ 0,55 s
Cải thiện	nhỏ hơn ~20×	nhANH hơn ~26×

⇒ Đây là bước “ai làm dữ liệu lớn cũng làm”: chuyển sang định dạng cột để truy vấn lặp lại tức thì. Ô chuyển đổi là *idempotent* — đã có thì bỏ qua (quy tắc *checkpoint* của dự án).

3.4 Bước 4 — Hồ sơ studentVle

Dữ liệu này trông như thế nào? **6.268** loại tài nguyên (`id_site`), **22** lượt mở môn. Quan trọng: chỉ **26.074** sinh viên có click — ít hơn 28.785 sinh viên trong `studentInfo`, nghĩa là **một số sinh viên không hề bấm gì** (tín hiệu nguy cơ!).

3.5 Bước 5 — Phân phối click & độ lệch phải → vì sao $\log_{1p}$

Gộp clickstream về hạt (môn, kỳ, sinh viên) để có `total_clicks`. So sánh **trung bình** với **trung vị**:

```
WITH agg AS (
  SELECT id_student, sum(sum_click) AS total_clicks
  FROM file('studentVle.parquet', 'Parquet')
  GROUP BY code_module, code_presentation, id_student)
SELECT round(avg(total_clicks),0) AS mean, quantileExact(0.5)(total_clicks) AS
  median,
  round(skewPop(total_clicks),2) AS skew, max(total_clicks) AS max_v FROM agg;
```

Kết quả: mean **1.355** nhưng median chỉ **740**; skew **2,94**; max **24.139**. Mean $\gg$ median và skew > 0 là dấu vân tay của **lệch phải nặng**: vài sinh viên siêu hoạt động kéo đuôi rất dài.

⇒ Đây là bằng chứng số cho quyết định $\log_{1p}$ ở bước làm sạch — nén đuôi mà vẫn giữ thứ hạng, không xóa dòng nào.

3.6 Bước 6 — Nhãn & cân bằng lớp

Đếm trực tiếp `final_result` trên `studentInfo`:

final_result	Số SV	Nhãn
Pass	12.361	không nguy cơ (0)
Withdrawn	10.156	nguy cơ (1)
Fail	7.052	nguy cơ (1)
Distinction	3.024	không nguy cơ (0)

Tỉ lệ nguy cơ = **52,8%** (tỉ số mất cân bằng chỉ 1,12 — lệch *nhẹ*). Đây là con số thật, không phải 68/32 như slide minh họa của môn. Lưu ý **Withdrawn** là lớp đơn lớn nhất — sẽ rất quan trọng ở phần EDA.

3.7 Bước 7 — Dữ liệu thiếu & cơ chế thiếu

Thiếu không chỉ là “đếm NaN” mà phải hỏi **vì sao thiếu** (cơ chế quyết định cách xử lý). Trong CSV, ô thiếu hiện ra dưới dạng chuỗi rỗng nên ta đếm = '':

Cột	% thiếu	Cơ chế & cách xử lý
date_unregistration	69,1%	Thiếu cấu trúc — chỉ tồn tại nếu sinh viên rút môn $\Rightarrow$ bỏ, không phải đặc trưng.
imd_band	3,41%	MAR/MCAR $\Rightarrow$ thêm hạng “Unknown” (không bịa mức thiết thời).
date_registration	0,14% (45 dòng)	MCAR $\Rightarrow$ điền median của tập train.

3.8 Bước 8 — Các bảng nhỏ còn lại (một câu mỗi bảng)

- **courses**: **22** lượt mở môn, độ dài **234–269** ngày (dùng để quy ngày $\rightarrow$ % tiến độ).
- **assessments**: **206** bài (TMA 106, CMA 76, Exam 24); 11 bài Exam không có hạn nộp.
- **studentAssessment**: **173.912** lượt nộp; 173 điểm thiếu.
- **vle**: **6.364** tài nguyên thuộc 20 loại hoạt động (nhóm giữ 8 loại chính).

3.9 Bước 9 — Ghép về hạt master ngay trong SQL

Câu hỏi quan trọng nhất: ghép **studentInfo** với **clickstream** đã gộp để ra hạt master.

```
WITH agg AS (
  SELECT code_module, code_presentation, id_student, sum(sum_click) AS total_clicks
  FROM file('studentVle.parquet', 'Parquet') GROUP BY 1,2,3)
SELECT count() AS master_rows,
       countIf(a.total_clicks = 0 OR a.total_clicks IS NULL) AS zero_click_rows,
       round(avg(ifNull(a.total_clicks,0)),1) AS mean_total_clicks
FROM file('studentInfo.csv', 'CSVWithNames', '...') AS si
LEFT JOIN agg AS a USING (code_module, code_presentation, id_student);
```

Kết quả: đúng **32.593** dòng, trong đó **3.365** dòng có **0 click** (sinh viên không hề hoạt động) $\Rightarrow$ sẽ được điền 0 ở bước làm sạch; trung bình **total_clicks** = 1.215,1 (khớp tuyệt đối với bảng master của pipeline).

3.10 Bước 10 — Cắt theo thời gian: trái tim của chống rò rỉ

Bài toán là **dự đoán sớm**: ở mốc $t\%$ khoá học ta chỉ được biết những gì xảy ra **trước** ngày cắt $\text{cutoff} = \text{round}(\text{độ_dài_môn} \times t/100)$. Đếm số sự kiện còn lại ở mỗi mốc:

Mốc $t\%$	10	20	40	60	80	100
Triệu sự kiện $\leq$ cutoff	2,72	4,12	5,96	7,93	9,43	10,66
% toàn bộ	25,5	38,7	56,0	74,4	88,5	100

$\Rightarrow$ Số sự kiện **chỉ tăng** theo t và mốc 100% giữ **trọn** 10,66 triệu — đúng như bộ test rò rỉ yêu cầu (không bản ghi nào vượt cutoff, số lượng không giảm theo t).

3.11 Bước 11 — Tín hiệu: yếu tố nào phân biệt nguy cơ, và sớm tới đâu?

Dùng bảng master đã dựng để đo **Cohen's d** — khoảng cách hai nhóm tính theo độ lệch chuẩn, công thức dự án dùng: $d = |m_1 - m_0| / \sqrt{(s_1^2 + s_0^2)/2}$.

Đặc trưng	TB không nguy cơ	TB nguy cơ	Cohen's d
days_since_last_activity	14,2	171,1	2,55
n_assessments_submitted	8,6	2,3	2,05
weighted_score_to_date	84,9	15,1	1,96
n_days_active	91,6	23,2	1,58

Tương quan kiểm rò rỉ: idle↔nhân **0,78**; active↔clicks 0,84; idle↔số bài nộp -0,83 — **không cặp nào** $\geq 0,95$ nên không có đặc trưng rò rỉ nhân. **Tín hiệu sớm** (Cohen's d theo mốc, cùng 32.593 sinh viên ở mọi mốc):

Mốc t%	10	20	40	60	80	100
n_days_active	0,83	0,99	1,17	1,37	1,50	1,58
mean_score_to_date	0,80	1,11	1,49	1,56	1,57	1,58
days_since_last_activity	0,91	1,17	1,56	1,92	2,30	2,55

⇒ *n_days_active* và *days_since_last_activity* đều đạt hiệu ứng lớn ($d \geq 0,8$) **ngay ở mốc 10%**; nhóm điểm ở 20%. **Tín hiệu hữu ích tồn tại rất sớm** — đó là cơ sở khoa học cho can thiệp sớm.

4 Từ dữ liệu đã hiểu → hoàn thành các task còn lại

Sau khi đã *hiểu* dữ liệu, các task còn lại không còn là “làm theo công thức” mà là **hệ quả tự nhiên** của những gì vừa thấy.

4.1 Làm sạch (Task 3)

- **Trùng lặp:** 0 dòng trùng trên khoá tổng hợp (kiểm bằng `drop_duplicates`).
- **Giá trị thiếu:** đúng theo cơ chế ở Bước 7 — “Unknown” cho `imd_band`; điền 0 + cờ `not_submitted` cho điểm; median train cho `date_registration`. Sau xử lý: **0 NaN** ở mọi cột đặc trưng.
- **Ngoại lai: không xoá dòng nào.** `log1p` cho mọi biến đếm click (vì lệch phải nặng, Bước 5); winsorize 1% cho biến có chặn (điểm, tín chỉ, số lần thi lại); để nguyên biến không có outlier thật (vd `mean_score_to_date`: biên IQR vượt cả 100 nên không có outlier).

Vì sao bước này? Mọi ngưỡng (median, giới hạn winsorize, scaler) đều **học chỉ trên tập train** rồi áp lên test — nguyên tắc vàng chống rò rỉ.

4.2 Biến đổi & chuẩn hoá (Tasks 16–20)

28 đặc trưng được chia **5 kiểu**, vì kiểu biến quyết định cách mã hoá:

Kiểu	Số biến	Bộ mã hoá / scaler
Định lượng (numeric)	19	StandardScaler (TB 0, DLC 1)
Thứ bậc (ordinal)	3	OrdinalEncoder (giữ thứ tự; giá trị lạ → -1)
Danh định (nominal)	3	OneHotEncoder (<code>drop=None</code> , <code>handle_unknown=ignore</code>)
Nhị phân (binary)	2	BinaryEncoder (M/Y → 1)
Chỉ báo (indicator)	1	passthrough (đã là 0/1)

One-hot mở rộng 3 biến danh định thành nhiều cột (region **13** + module **7** + kỳ **4**). Tổng cộng: $19 + 3 + 2 + 1 + 13 + 7 + 4 =$ **49 cột** đặc, không NaN.

⇒ *drop=None* (giữ mọi cột one-hot) là chủ ý: đây là đồ án giải thích được — bỏ một hạng tham chiếu sẽ giấu nó khỏi SHAP/LIME.

4.3 Chia tập & chống rò rỉ (Tasks 7, 8, 15, 22)

Việc chia tập tôn trọng **ba** tính chất của dữ liệu cùng lúc:

1. **Theo nhóm** — một sinh viên có thể xuất hiện ở nhiều lượt mở môn $\Rightarrow$ chia theo `id_student` (nếu không, cùng một người rơi vào cả train lẫn test = rò rỉ nhóm).
2. **Phân tầng** — giữ tỉ lệ nguy cơ 52,8% ở cả hai tập.
3. **Theo thời gian** — test **giống hệt** ở mọi mốc để so sánh công bằng.

Kết quả (hold-out 20%, seed 42): **26.104** train / **6.489** test / **5.756** sinh viên test; tỉ lệ nguy cơ 0,53 vs 0,52; **0** sinh viên trùng. Và chống rò rỉ **được kiểm thử tự động**: `tests/test_leakage.py` chạy **19/19 test pass** — gồm test train-only cho điền khuyết/winsorize, test allow-list chống đưa cột rò rỉ vào X, và test khớp idle ở t=100% với master.

4.4 Phân tích khám phá — EDA (Tasks 27–38)

Những phát hiện chính (đã định lượng ở Bước 5–11):

- Phân bố lệch phải mạnh $\Rightarrow$ `log1p`; `days_since_last_activity` **lượng đỉnh** — chữ ký của mất gắn kết (median nhàn rỗi: Pass 12, Fail 116, Withdrawn 232 ngày).
- **Hành vi áp đảo nhân khẩu**: Cohen's d tới 2,55 so với Cramér's $V \leq 0,15$.
- Có 2 cặp cộng tuyến ($\geq 0,8$) nhưng không cặp nào $\geq 0,95 \Rightarrow$ không rò rỉ.
- Giả thuyết cho mô hình: *mới ngừng hoạt động gần đây + nộp ít bài* là dấu hiệu nguy cơ sớm và đáng tin nhất.

5 Tái lập toàn bộ (chạy lại từ đầu)

```
# 0) Understand the data (this notebook -- ClickHouse, run step by step):
#    open notebooks/00_data_understanding.ipynb, Run All (kernel dsp)

# 1) Build the master table + 6 checkpoints from the 7 raw CSVs:
python -m src.data.build_master_table
python -m src.data.make_checkpoints          # idempotent, resumable

# 2) Fixed train/test split + anti-leakage report:
python -m src.evaluation.make_split

# 3) Leakage tests (must show 16 passed):
pytest tests/test_leakage.py -q

# 4) Generate EDA figures + tables, then notebooks 01/02:
python -m src.eda.eda
python tools/build_notebooks.py

# 5) Compile the documents (Tectonic):
tectonic reports/guide/DataTask_Process_Guide.tex
tectonic reports/slides/DataTasks_Slides.tex
```

Phụ lục — Bảng tổng hợp con số (đã kiểm chứng)

Hạng mục	Giá trị
studentVle (click-stream)	10.655.280 dòng · 433 MB · 26.074 SV có click
Parquet (sau chuyển)	22,9 MB · truy vấn < 1 s
Hạt master	32.593 dòng · 28.785 SV · 22 lượt mở môn · 33 cột
Nhãn nguy cơ	52,8% (tỉ số 1,12) · Pass 12.361 / Withdrawn 10.156 / Fail 7.052 / Dist. 3.024
Thiếu	imd_band 3,41% · date_registration 0,14% · date_unregistration 69,1%
Lệch phải	total_clicks skew 3,0 · max_clicks_single_day skew ≈ 11
Mã trận đặc trưng	28 đặc trưng (5 kiểu) $\rightarrow$ 49 cột
Chia tập	26.104 train / 6.489 test / 5.756 SV test · 0 trùng · 19/19 test rõ rĩ
Hiệu ứng mạnh nhất	days_since_last_activity Cohen's d = 2,55 · sớm từ mốc 10%

Tài liệu tham khảo. [1] Kuzilek, Hlosta, Zdrahal, *Open University Learning Analytics Dataset*, Scientific Data 4:170171, 2017. [2] Adnan et al., *IEEE Access* 9:7519–7539, 2021. [3] Tomasevic, Gvozdenovic, Vranes, *Computers & Education* 143:103676, 2020.